

Discussion 1: Data Structures

Discussion Topic:

How do different data structures solve specific problems in modern applications, and why is understanding their importance crucial for effective software design? Use a specific example from a real application (streaming service, social media, navigation app, etc.) and discuss how algorithm study enhances problem-solving approaches in that context.

My Post:

Hello class,

To discuss how several data structures and algorithms must work together to solve problems, I chose to use the Google Maps app.

The road network feature of the app uses a graph data model, which is a data structure that uses vertices and edges to illustrate relationships between various locations.

In this model, every intersection is a vertex, and every road segment is a weighted edge. A weighted edge is an edge that is assigned a specific cost, a weight based on variables such as distance or travel time.

The app's search feature uses a hash table to retrieve location data. For example, when a user types a store name, the app finds the store location by matching the query against millions of stored locations almost instantly; this is handled by the hash table. The table stores items in an unordered items and maps each of them to a location in an array.

To compute the fastest route, the app uses Dijkstra's shortest path algorithm, which has a near-constant-time $O(1)$. The algorithm requires a priority queue to be implemented with a min-heap, which is a tree that stores nodes' properties such that the node's key is less than or equal to its children's keys. This ensures that the closest vertex is always processed first. In this example, it finds the shortest path from an initial or start vertex to each vertex in the graph.

The Google Maps app uses each data structure for a specific purpose, the graph models relationships, the hash table provides fast retrieval, and the heap manages processing order.

Table 1

Dijkstra - Priority Queue

Priority Queue Implementation	Time Complexity
Unsorted Array	$O(V^2)$
Binary Heap (Min-Heap)	$O((V + E) \log V)$ Often simplified to $O(E \log V)$ for connected graphs.
Fibonacci Heap	$O((E + V) \log V)$

Note: V be the number of vertices (nodes), and E be the number of edges.

The course textbook explains that choosing the right algorithm is essential for building effective and performant software applications (Lysecky & Vahid., 2019). For example, a linear search through 200 million items could take over 3 minutes, while a binary search on sorted data finishes in under 28 μ s, up to 7,000,000 times faster. The Google Maps app developer faced a similar challenge, as

implementing a brute-force search feature of every possible route in a city network would have an exponential $O(2^N)$ time complexity, making real-time navigation impossible. On the other hand, binary search has $O(\log N)$ time complexity by repeatedly dividing the search space in half. This “divide and conquer” principle applies to Dijkstra’s algorithm, with its $O((V + E) \log V)$ time complexity, see Table 1, making the algorithm practical to use at the city or even global scale.

As shown by this small time complexity algorithm analysis example, an algorithm study provides software developers with an analytical perspective of the efficiency of their software. In other words, instead of guessing whether a solution will be fast enough or if it will consume too much memory, developers can use Big O notation or space complexity analysis to mathematically evaluate growth rates and predict how the performance of the app will be as the input size increases.

The Google Maps app example showcases well why understanding data structures is essential for software developers, as it allows them to organize, manage, and process data as efficiently as possible. Additionally, choosing the best data structure for a specific software project requires determining which structure provides the best balance between time and space complexity, based on the expected uses and technical constraints. It also highlights that choosing the right algorithms is often dependent on the data structures they must operate on. Ultimately, a developer’s ability to implement the right data structures with the correct algorithms dictates whether an application will be responsive and reliable for the end user.

-Alex

References:

Lysecky, R., & Vahid, F. (2019, August). Section 2.2 Binary search. Design and analysis of algorithms. In R. Lysecky, & F. Vahid, *Data structures essential: Pseudocode with python examples*. Zybooks.